

Course Outline: Files in Python

Title: Working with Files in Python **Learnpack:** + Trabajar con ficheros en Python **Prerequisite:** Intro to Python (Week 7), Building a Python API (Week 8 Day 22), Basic Agents in Python (Week 8 Day 23) **Target Audience:** Beginner developers who already know Python fundamentals (variables, control flow, functions, lists, dictionaries) and have built basic APIs with FastAPI **Total Lessons:** 10 **Estimated Total Length:** ~5,000 words **Format:** Conceptual (0% hands-on)

Course Description

This course teaches students how to work with files in Python — reading, writing, and managing persistent data on disk. Until now, students have worked with data that lives only in memory (variables, lists, dictionaries). Files are the first step toward data persistence: saving information so it survives after the program stops running.

Students will learn how to open, read, and write text files using Python's built-in tools, understand file encoding and path resolution, and work with CSV files as a bridge between raw text and structured data (lists of dictionaries). This learnpack sets the foundation for the next skill (Week 9, Day 25), where students will move from file-based storage to database-based storage with TinyDB.

Course Philosophy

- This course emphasizes:
- **Path of least resistance:** Start with the simplest file operations (`with open(...)`) before introducing CSV specifics.
 - **Attention to detail:** File operations are unforgiving — a wrong mode (`"w"` instead of `"a"`) can destroy data. Students must read carefully.
 - **Practical context:** Every concept connects to real backend scenarios — persisting user data, loading configuration, exporting reports.
-

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - Files and Paths	2	~1,000
02 - Reading and Writing	3	~1,500
03 - CSV Files	2	~1,100
04 - Assessment	1	~700
05 - Wrap-Up	1	~450

Section	Lessons	Estimated Words
Total	10	~5,200

Section 00: Welcome

00.0 Welcome 📖 (~450 words)

Learning Objectives:

- Understand what this course covers and why file operations matter in backend development
- Recognize the gap between in-memory data and persistent data

Content Outline:

- The problem: everything you've built so far loses its data when the program stops
- Files as the simplest form of persistence — data that survives between runs
- What you'll learn: reading files, writing files, working with CSV as structured data
- How this connects to databases (coming in the next skill with TinyDB)
- Brief overview of the course structure

Transitions: This lesson frames the "why" behind file I/O. The next section dives into the fundamentals: what a file actually is, how encoding works, and how Python locates files on disk.

Key Principles:

- Data in variables is temporary; data in files is persistent
- Files are the simplest persistence mechanism — understand them before moving to databases

Examples:

- A FastAPI endpoint that receives user data but loses it on restart — the motivation for persistence
 - A simple analogy: variables are like a whiteboard (erased when you leave), files are like a notebook (stays on the shelf)
-

Section 01: Files and Paths

01.0 What Is a File? 📖 (~500 words)

Learning Objectives:

- Define what a file is in the context of a computer's file system
- Understand that files store bytes on disk and programs interpret those bytes

Content Outline:

- A file is a named sequence of bytes stored on disk
- Text files vs binary files (this course focuses on text files)
- Files have a name, an extension (`.txt`, `.csv`, `.py`), and a location (path)
- The operating system manages files; Python asks the OS to open, read, or write them

- The lifecycle: open → read or write → close

Transitions: Now that you know what a file is, the next lesson covers two critical details: how text is encoded inside files, and how Python finds files using paths.

Key Principles:

- A file is just bytes on disk — the program decides how to interpret them
- Always close what you open (or better: let Python close it for you)

Examples:

- A `.txt` file containing `"Hello"` is stored as specific bytes (72, 101, 108, 108, 111 in ASCII)
 - The same bytes interpreted differently could look like gibberish — that's an encoding mismatch
-

01.1 Encoding and Paths 📖 (~500 words)

Learning Objectives:

- Explain what file encoding is and why UTF-8 is the default choice
- Distinguish between absolute and relative paths and know when to use each

Content Outline:

- Encoding: how characters are translated to bytes and back
 - UTF-8 as the universal default — handles virtually all languages and symbols
 - What happens when encoding is wrong: garbled text (mojibake)
 - Always specify `encoding="utf-8"` explicitly in Python
- Paths: how Python finds your file
 - Absolute paths: the full address from the root (`/home/user/data/file.txt`)
 - Relative paths: the address from where your script runs (`./data/file.txt`)
 - The concept of the working directory — "where am I right now?"
 - Using `os.path.exists()` to check if a file exists before trying to read it

Transitions: With encoding and paths understood, you're ready to actually open and read files in the next section.

Key Principles:

- Always specify `encoding="utf-8"` — never rely on system defaults
- Know where your script runs from; relative paths depend on it
- Validate that a file exists before trying to open it (`os.path.exists()`)

Examples:

- Opening a file with the wrong encoding: `"café"` becomes `"café"`
 - A script in `/home/user/app/` trying to read `../data/users.txt` — tracing the relative path step by step
-

Section 02: Reading and Writing

02.0 Open and Read 📖 (~500 words)

Learning Objectives:

- Use `with open(...)` to safely open and read text files in Python
- Choose between `read()`, `readline()`, and iterating line by line

Content Outline:

- The `with open(...)` pattern: why it's the only way you should open files
 - Automatically closes the file, even if an error occurs
 - Syntax: `with open("file.txt", "r", encoding="utf-8") as f:`
- Reading the entire file at once: `f.read()` — returns a single string
- Reading one line at a time: `f.readline()` — useful for processing line by line
- Iterating over lines: `for line in f:` — the most Pythonic and memory-efficient way
- The `"r"` mode: read-only (default)
- Stripping newlines: `line.strip()` to remove trailing `\n`

Transitions: Reading is only half the story. The next lesson covers how to create new files and write data to them.

Key Principles:

- Always use `with open(...)` — never use bare `open()` without a context manager
- For large files, iterate line by line instead of loading everything into memory
- `encoding="utf-8"` should always be specified explicitly

Examples:

- Reading a small configuration file entirely with `f.read()`
 - Processing a log file line by line with `for line in f:` — printing only lines that contain the word `"ERROR"`
-

02.1 Create and Write 📖 (~500 words)

Learning Objectives:

- Create new files and write text content to them using Python
- Distinguish between write mode (`"w"`) and append mode (`"a"`)

Content Outline:

- Writing to a file: `with open("file.txt", "w", encoding="utf-8") as f:`
- The `"w"` mode: creates the file if it doesn't exist, **overwrites** if it does
- The `"a"` mode: creates the file if it doesn't exist, **appends** to the end if it does
- Writing content: `f.write("some text\n")` — you must add `\n` yourself
- Writing multiple lines: `f.writelines(list_of_strings)`
- The importance of `\n`: without it, all your text ends up on one line
- Practical pattern: read data → process it → write the result to a new file

Transitions: Now that you know how to read and write, the next lesson covers the most common mistakes developers make with file operations — and how to avoid them.

Key Principles:

- `"w"` mode is destructive — it erases existing content. Use it intentionally.
- `"a"` mode is safe for adding data without losing what's already there
- Always add `\n` at the end of each line you write

Examples:

- Creating a simple text report: opening a new file, writing a header line, then writing data rows
- The difference between running a script twice with `"w"` (only the last run's data survives) vs `"a"` (both runs' data is preserved)

02.2 Writing Pitfalls 📖 (~500 words)

Learning Objectives:

- Identify the most common mistakes when working with files in Python
- Apply defensive patterns to avoid data loss and resource leaks

Content Outline:

- **Anti-pattern 1: `open()` without `with`** — if an error occurs, the file stays open ("dangling file handle"). The OS may lock the file or buffer data that never gets written.
 - Correct approach: always use `with open(...) as f:`
- **Anti-pattern 2: Using `"w"` when you meant `"a"`** — accidentally overwriting a file full of important data. This is silent and irreversible.
 - Correct approach: pause and think about intent before choosing the mode. Default to `"a"` if unsure.
- **Anti-pattern 3: Forgetting `\n`** — all output ends up glued on a single line: `"Alice25Bob30Carol22"` instead of separate lines.
 - Correct approach: always include `\n` or use formatted strings like `f"{name}\n"`
- **Anti-pattern 4: `readlines()` on large files** — loads the entire file into memory as a list. A 2GB log file will crash your program.
 - Correct approach: iterate with `for line in f:` which processes one line at a time
- Summary: a quick checklist before any file operation (Am I using `with`? Is my mode correct? Am I handling newlines? Is the file small enough to read entirely?)

Transitions: You now have solid foundations for working with plain text files. The next section introduces CSV files — a structured format that bridges the gap between raw text and Python data structures like lists and dictionaries.

Key Principles:

- File operations fail silently in dangerous ways — a wrong mode doesn't throw an error, it just destroys your data
- Defensive coding matters more with files than almost anywhere else
- When in doubt: use `with`, use `"a"`, add `\n`, iterate line by line

Examples:

- Side-by-side comparison: a script with bare `open()` that leaks file handles vs the same script with `with open(...)`
 - A real scenario: a developer runs their export script twice with `"w"` and loses the first batch of data
-

Section 03: CSV Files

03.0 CSV Structure 📖 (~500 words)

Learning Objectives:

- Explain what a CSV file is and why it's commonly used for structured tabular data
- Identify the components of a CSV file (headers, rows, delimiters)

Content Outline:

- CSV stands for "Comma-Separated Values" — it's a plain text file where each line is a row and values are separated by commas
- The first line is typically the header row (column names)
- Each subsequent line is a data row
- Why CSV matters: it's the universal exchange format for tabular data — spreadsheets, databases, APIs, and data pipelines all speak CSV
- CSV limitations: no data types (everything is text), no nesting, commas inside values require quoting
- Python's built-in `csv` module: `csv.reader` and `csv.DictReader`
- Quick comparison: a CSV file vs the same data as a list of dictionaries in Python — they represent the same thing

Transitions: Now that you understand the structure, the next lesson shows you how to convert between CSV files and Python data structures in both directions.

Key Principles:

- CSV is just structured text — each line is a record, each comma is a field separator
- The header row defines the "keys"; each data row provides the "values"
- Python's `csv` module handles edge cases (quoted fields, commas inside values) so you don't have to

Examples:

- A CSV file with three columns (`name, age, email`) and three data rows
 - The same data represented as a Python list of dictionaries: `[{"name": "Alice", "age": "25", "email": "alice@example.com"}, ...]`
-

03.1 CSV to Dicts and Back 📖 (~600 words)

Learning Objectives:

- Read a CSV file and convert it into a list of dictionaries using `csv.DictReader`
- Write a list of dictionaries back to a CSV file using `csv.DictWriter`

Content Outline:

- **Reading CSV → List of Dicts:**
 - `csv.DictReader` reads each row as a dictionary where keys come from the header row
 - Pattern: `with open("data.csv", "r", encoding="utf-8") as f: → reader = csv.DictReader(f) → data = list(reader)`
 - Result: a Python list of dictionaries — easy to iterate, filter, and manipulate
 - Each value is a string by default — cast to `int` or `float` if needed
- **Writing List of Dicts → CSV:**
 - `csv.DictWriter` takes a list of field names and writes dictionaries as rows
 - Pattern: define `fieldnames`, open file with `"w"`, create writer, call `writer.writeheader()`, then `writer.writerows(data)`
 - The `fieldnames` parameter controls which keys are written and in what order
- **Round-trip pattern:** read a CSV → modify the data in Python → write it back to a new CSV
- Why this matters for backend development: loading seed data, exporting reports, processing uploads

Transitions: You've now covered all the core topics of this learnpack: file fundamentals, reading, writing, and CSV operations. The next section tests your understanding with a knowledge check.

Key Principles:

- `csv.DictReader` turns CSV rows into dictionaries — the most natural Python representation of tabular data
- `csv.DictWriter` turns dictionaries back into CSV rows — always define `fieldnames` explicitly
- The round-trip (CSV → dicts → CSV) is a fundamental data processing pattern in backend development

Examples:

- Reading a `users.csv` file into a list of dictionaries, then printing only users whose age is above 18
- Taking a list of product dictionaries and writing them to a `products_export.csv` file with specific column ordering

Section 04: Assessment

04.0 File Operations Check 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of file reading and writing patterns in Python
- Identify correct and incorrect file handling practices
- Apply CSV reading and writing knowledge to practical scenarios

Question Format: Scenario-based (7 questions)

Questions:

Q1: You need to read a configuration file that is only 50 lines long. Which approach is most appropriate?

- A) `f.read()` to load the entire file into a string
- B) `for line in f:` to iterate line by line

- C) `f.readlines()` to get a list of all lines
- D) All three work fine for a small file like this

Correct: D — For a 50-line file, all three approaches are functionally equivalent and safe. Memory is not a concern at this size.

Q2: A developer writes this code: `open("log.txt", "w", encoding="utf-8").write("New entry\n")`. What are the two problems?

- A) The file won't be created because `"w"` mode doesn't create files
- B) The file is never explicitly closed, risking a dangling file handle
- C) The `encoding` parameter is not valid for write mode
- D) Using `"w"` will erase any existing content in `log.txt`

Correct: B and D — No `with` statement means the file may not be properly closed, and `"w"` mode overwrites existing content.

Q3: You have a 3GB server log file and need to find lines containing `"ERROR"`. Which approach should you use?

- A) `data = f.read()` then search the string
- B) `lines = f.readlines()` then filter the list
- C) `for line in f:` and check each line
- D) Use `csv.DictReader` to parse the log

Correct: C — Iterating line by line processes one line at a time without loading the entire file into memory.

Q4: What does `encoding="utf-8"` do when opening a file?

- A) Compresses the file to take less disk space
- B) Tells Python how to translate between bytes on disk and characters in your string
- C) Encrypts the file content for security
- D) Converts the file from binary to text format

Correct: B — Encoding defines the mapping between raw bytes and human-readable characters.

Q5: You run an export script that uses `open("report.csv", "w")` twice in a row. What happens to the data from the first run?

- A) It's preserved; the second run appends to it
- B) It's completely erased; only the second run's data remains
- C) Python raises a `FileExistsError`
- D) Both runs' data is merged automatically

Correct: B — `"w"` mode truncates the file on every open, so the first run's data is lost.

Q6: You read a CSV file with `csv.DictReader`. The CSV has a column `"age"` with value `"25"`. What is the Python type of that value?

- A) `int`
- B) `float`
- C) `str`

- D) It depends on the CSV content

Correct: C — `csv.DictReader` always returns string values. You must explicitly cast to `int` or `float` if needed.

Q7: You need to write a list of dictionaries to a CSV file. You use `csv.DictWriter` but forget to call `writer.writeheader()`. What happens?

- A) Python raises an error
- B) The CSV file is created with data rows but no header row
- C) The first dictionary's keys are automatically used as headers
- D) The file is empty

Correct: B — The data rows are written normally, but the file has no header row, making it harder to interpret.

Evaluation Criteria:

- 6-7 correct: Excellent — ready for the next skill
- 4-5 correct: Good — review the specific topics you missed
- 0-3 correct: Review the full course before proceeding

Section 05: Wrap-Up

05.0 What's Next (~450 words)

Content Outline:

- Course recap: you've learned how to persist data beyond your program's lifetime using files
 - Opening and reading text files safely with `with open(...)`
 - Writing and appending to files while avoiding common pitfalls
 - Working with CSV files as structured data — converting between CSV and Python dictionaries
- Connection to the bigger picture: files are the simplest form of data persistence, but they have limitations — no querying, no concurrent access, no relationships between data. That's why databases exist.
- What's next: in the next skill (Week 9, Day 25), you'll move from file-based storage to TinyDB — a lightweight database that lets you persist, query, and update structured data from your FastAPI application.
- The skills you learned here (reading CSVs, understanding encoding, handling paths) will come back when you need to load seed data into your database, export reports, or process file uploads.
- Keep practicing: try reading and writing different file formats, experiment with error scenarios (wrong path, wrong mode, wrong encoding), and build confidence with the `csv` module.

Note: This is conclusion only — no theory, exercises, or assessment.